

OICPC455

AY 2024-25

Machine Learning Lab

Quality Laboratory Manual

Experiment No. 9

Study and Implementation of a Neural Network



Course Instructor -
DR. TAHSEEN A. MULLA
ASSOCIATE PROFESSOR

Experiment No. 9

Title of Experiment: To study and implement a Neural Network

Aim of Experiment: To implement and understand the working of a simple neural network for binary classification focusing on its structure, training process and evaluation

System Requirements – Win 8 and above OS, 4GB RAM, 2.33 GHz Processor

Software/s Needed for Experiment – Jupyter Notebook/ Anaconda Navigator/ Google Colaboratory/ Spyder, Python 3.x [With libraries such as Numpy, Pandas, Matplotlib, Scikit-Learn and Tensorflow/Keras]

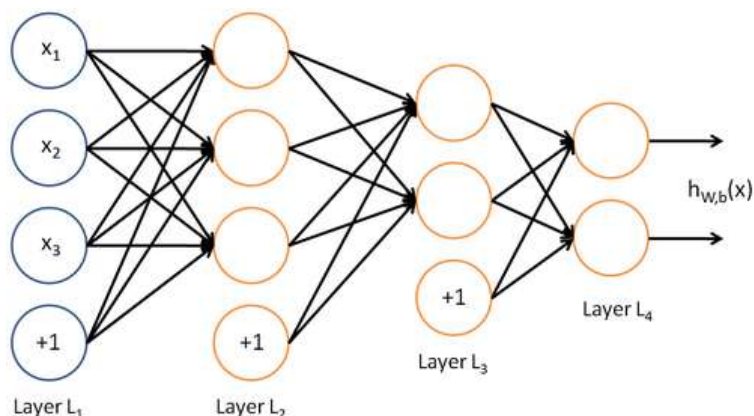
Experiment Outcomes –

1. Understand the structure of Neural Networks
2. To choose appropriate activation functions
3. To train and evaluate a Neural Network
4. To apply Neural Networks to real-world applications

Theory –

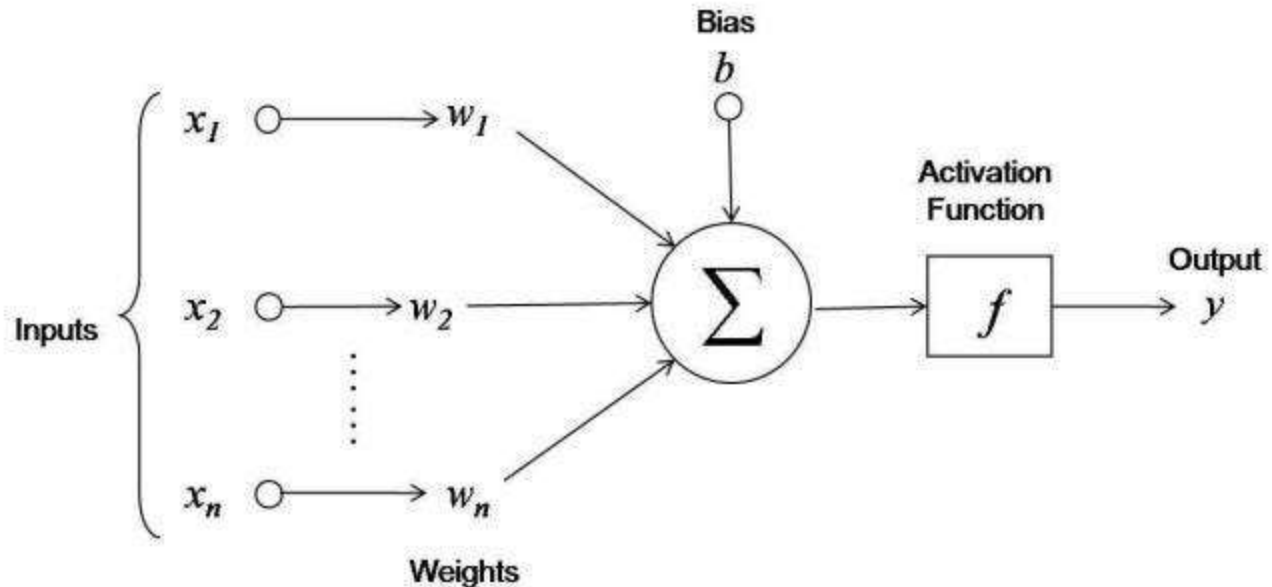
A Neural Network is a computational model inspired by the human brain, made up of layers of interconnected "neurons." Each neuron processes input data, applies weights and biases, and passes the result through an activation function. Neural networks learn by adjusting weights to minimize the error in predictions.

Based on nature, neural networks are the usual representation we make of the brain: neurons interconnected to other neurons which forms a network. A simple information transits in a lot of them before becoming an actual thing, like “move the hand to pick up this pencil”



Neural networks can usually be read from left to right. Here, the first layer is the layer in which inputs are entered. There are 2 internal layers (called hidden layers) that do some math, and one last layer that contains all the possible outputs

What does a Neuron do?

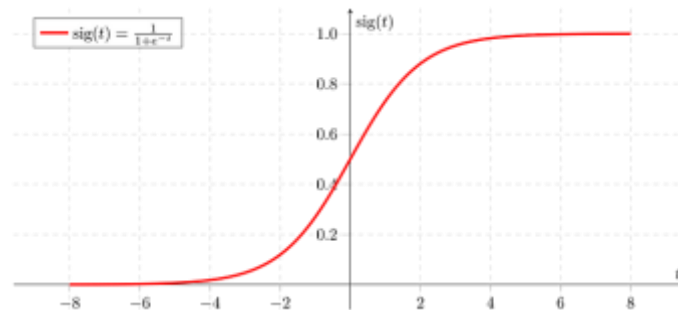


First, it adds up the value of every neuron from the previous column it is connected to. In the above figure, there are 3 inputs (x_1, x_2, x_3) coming to the neuron, so 3 neurons of the previous column are connected to our neuron

This value is multiplied, before being added, by another variable called “weight” (w_1, w_2, w_3) which determines the connection between the two neurons. Each connection of neurons has its own weight, and those are the only values that will be modified during the learning process

Moreover, a bias value may be added to the total value calculated. It is not a value coming from a specific neuron and is chosen before the learning phase, but can be useful for the network

After all those summations, the neuron finally applies a function called “activation function” to the obtained value



The so-called activation function usually serves to turn the total value calculated before to a number between 0 and 1 (done for example by a sigmoid function shown in the figure). Other function exists and may change the limits of our function, but keeps the same aim of limiting the value

That's all a neuron does! Take all values from connected neurons multiplied by their respective weight, add them, and apply an activation function. Then, the neuron is ready to send its new value to other neurons

After every neurons of a column did it, the neural network passes to the next column. In the end, the last values obtained should be one usable to determine the desired output

Key Concepts –

- **Input Layer:** Takes the feature data as input
- **Hidden Layers:** Process inputs through interconnected neurons and apply non-linear transformations
- **Output Layer:** Provides the final prediction. For binary classification, it outputs values between 0 and 1
- **Activation Function:** Introduces non-linearity, allowing the network to learn complex patterns. Common functions include ReLU, Sigmoid, and Softmax
- **Loss Function:** Measures the difference between predictions and actual values (e.g., Binary Cross-Entropy for binary classification)
- **Optimizer:** Adjusts weights and biases to minimize the loss (e.g., Stochastic Gradient Descent or Adam)

Procedure to implement a Neural Network:**1. Data Collection and Preparation:**

- Use a sample dataset such as Iris or Breast Cancer (For binary classification select only two classes)
- Pre-process the data – clean, normalize and split into training and testing sets

2. Defining the Neural Network Model:

- Use the Keras sequential API to create a neural network model
- Add an input layer matching the number of features in the dataset
- Add one or more hidden layers with a suitable number of neurons and activation functions

- Add an output layer with a single neuron and sigmoid activation for binary classification

3. Compiling the Model:

- Set the loss function to Binary Cross-Entropy for binary classification
- Choose an optimizer (such as Adam or SGD) and define metrics like accuracy for evaluation

4. Training the Model:

- Fit the model on the training data for a specified number of epochs
- Track the loss and entropy across epochs to observe the model's learning process

5. Evaluate the Model:

- Use the test dataset to evaluate the model's performance
- Calculate metrics such as accuracy, precision, recall and F1-score to assess classification effectiveness

6. Visualize Training History:

- Plot the loss and accuracy over epochs to analyze training and validation performance

Sample Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import BinaryCrossentropy
```

Step 1: Data Collection and Preparation

Using a built-in dataset from Scikit-learn

```
from sklearn.datasets import load_breast_cancer
data = load_breast_cancer()
X = data.data
y = data.target
```

Data Preprocessing: Standardization and Train-Test Split

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
                                                    test_size=0.2, random_state=42)
```

Step 2: Define the Neural Network Model

```
model = Sequential([
    Dense(16, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid') # Output layer for binary
classification
])
```

Step 3: Compile the Model

```
model.compile(optimizer=Adam(learning_rate=0.001),
              loss=BinaryCrossentropy(),
              metrics=['accuracy'])
```

Step 4: Train the Model

```
history = model.fit(X_train, y_train, epochs=50, batch_size=16,
                    validation_split=0.2)
```

Step 5: Evaluate the Model

```
y_pred = (model.predict(X_test) > 0.5).astype("int32")
print("\nClassification Report:\n", classification_report(y_test,
y_pred))
print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

Step 6: Visualize Training History

```
plt.figure(figsize=(12, 4))
```

Plotting Loss

```
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title('Training and Validation Loss')
```

Plotting Accuracy

```
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation
Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training and Validation Accuracy')
plt.show()
```

Observations -

- Track the model's training and validation accuracy over epochs
- Note the final accuracy on the test set and compare with training performance to evaluate generalization

Conclusion –

Hence, the model summarizes the performance of the neural network and discuss improvements such as adding more layers, adjusting hyper-parameters or using regularization techniques

References –**a. Textbook –**

- i. Machine Learning with Python – An approach to Applied ML – Abhishek Vijayvargiya, BPB Publications, 1st Edition 2018
- ii. Machine Learning, Tom Mitchell, McGraw Hill Education, 1st Edition 1997

b. Online references –

- i. <https://mize.tech/blog/how-does-a-neural-network-work-implementation-and-5-examples/>
- ii. <https://towardsdatascience.com/first-neural-network-for-beginners-explained-with-code-4cfd37e06eaf>
- iii. https://medium.com/@mullatahseen_786/understanding-the-perceptron-foundations-of-neural-networks-in-deep-learning-b1b5e98c6657

Expected Oral Questions –

1. What is a Neural Network and how does it work?
2. Explain the difference between an artificial neuron and a biological neuron?
3. What are the main components of a neural network? Describe their roles
4. Define the terms input layer, hidden layer and output layer in a neural network?
5. What is an activation function? Why is it important in a neural network?
6. How do neural networks learn from data? Briefly describe the learning process
7. What is the purpose of the loss function in training a neural network? Provide examples of common loss function
8. Describe the role of an optimizer in a neural network
9. What are hyper-parameters in a neural network? Provide examples and explain their impact on model performance
10. Draw a neat and labelled diagram of a neural network and explain its each entity

FAQ's in Interview –

1. How are Neural Networks modelled?
2. How do Neural Networks get the optimal weights and bias values?
3. What is an Activation function?
4. Explain the working of a Perceptron
5. Explain why the initialization process of weights and bias is important for Neural Networks

